Distributed Todo Application - VS Lab Project

Documentation

Ziyi Hong, Danylo Tulainov

2025-09-26

Contents

Project Documentation Overview

Distributed Systems Lab Project

Team Members: Ziyi Hong, Danylo Tulainov **Roles**: Frontend developer(Danylo), Backend developer(Ziyi)

GitHub Repository: https://github.com/VerSysLabTin23/TodolistProject

All documentation is available in the `docs/` folder of the repository.

Project Summary

This project implements a distributed, real-time collaborative Todo application using microservices architecture. The system features team-based task management, real-time updates via WebSocket, and event-driven communication using Apache Kafka.

Documentation Structure

Getting Started

- Setup Guide - Quick start and installation instructions

Architecture & Design

- Architecture Implementation - Comprehensive architecture documentation

Project Reflection

- Backend Reflection - Project challenges, learnings, and improvements

Backend Services

- Backend Overview - Backend services and components
  - Auth Service - Authentication and user management
  - Task Service - Task operations and event publishing
  - Team Service - Team management and permissions
  - Realtime Service - WebSocket and real-time updates
  - Notification Service - Email notifications
  - Kafka Integration - Event streaming and partitioning
  - Nginx Gateway - API gateway and load balancing

Frontend

- Frontend Overview - React application architecture

Key Features

- **Microservices Architecture**: Independent services for scalability
- **Event-Driven Communication**: Kafka-based asynchronous messaging
- **Real-time Collaboration**: WebSocket-powered live updates
- **Team-based Organization**: Multi-user task management
- **Horizontal Scaling**: Load-balanced service deployment
- **Containerized Deployment**: Docker-based infrastructure

Technology Stack

Backend: - Go 1.21+ with Gin framework - MySQL 8.0 (database per service) - Apache Kafka for event streaming - JWT authentication

Frontend: - React 18 with TypeScript - WebSocket for real-time updates - Vite build tool

Infrastructure: - Docker & Docker Compose - Nginx reverse proxy - Mailpit for email testing

Quick Links

- **Start Here**: Setup Guide
- **API Reference**: API Endpoints
- **Architecture**: Architecture Implementation
- **Backend Services**: Backend Overview
- **Project Reflection**: Backend Reflection
- **Development_related**: Branching Strategy
- **Frontend**: Frontend Overview

Setup Guide

Prerequisites: Docker

Quick Start

1. Setup Environment

```
# Clone repository
git clone <repository-url>
cd TodolistProject

# Create environment file
cp .env.example .env

# Edit if needed (optional)
nano .env
```

2. Start Application

```
# Start all services
docker-compose up -d

# Check status
docker-compose ps
```

3. Access Application URL: http://localhost

Default Users for login

Username	Password
admin	password
john_doe	password
jane_smith	password

Architectur and Implementation

1. Architektur

Diagram

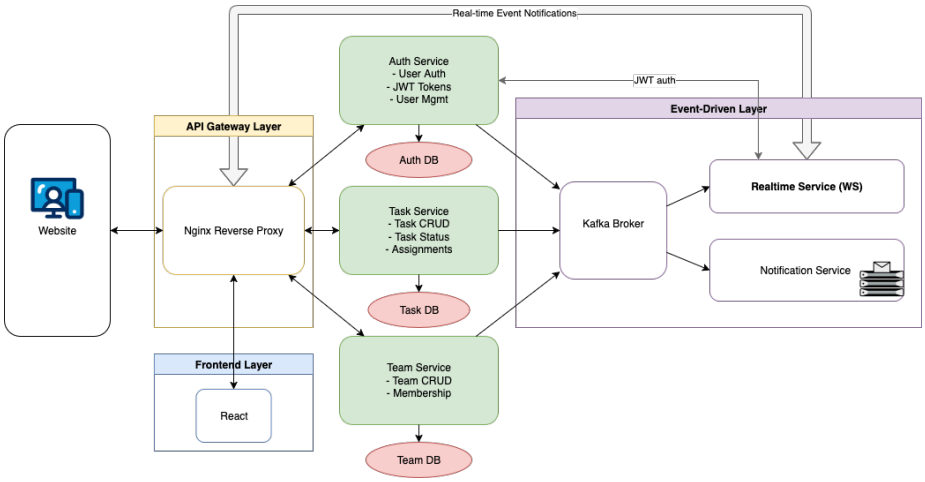

Figure 1: new_architecture

1.1 General Overview

Solution
This project is a distributed Todo application system designed with a microservices architecture.
The initial design followed a pure microservices approach, but later—due to requirements from another course (Verteilte Systeme)—we extended the system with Apache Kafka. This addition introduced an event-driven component, where Apache Kafka serves as the message middleware to enable asynchronous communication between the backend core business services (task,team,auth services), realtime service and notification service. Kafka was implemented after the development of core business services. So it doesn't support the communication between the three core business services. Those three core business services still communicate with API request.

Core Architecture Features:
Microservices Architecture: Decomposed monolithic application into independent services (Auth, Team, Task, Realtime, Notification)
Event-Driven: Using Kafka for loose coupling between (some) services
Real-time Communication: Providing real-time updates through WebSocket
Containerized Deployment: Using Docker and Docker Compose for service orchestration
Load Balancing: Implementing API gateway and load balancing through Nginx

1.2 Key Architectural Decisions

- Microservices: Independent services (Auth, Team, Task, Realtime, Notification) for scalability and maintainability.
- Kafka: High-throughput message broker for event streaming and persistence.
- WebSocket: Real-time bidirectional communication for collaborative features.
- Seperate Database: Data isolation and independent scaling.

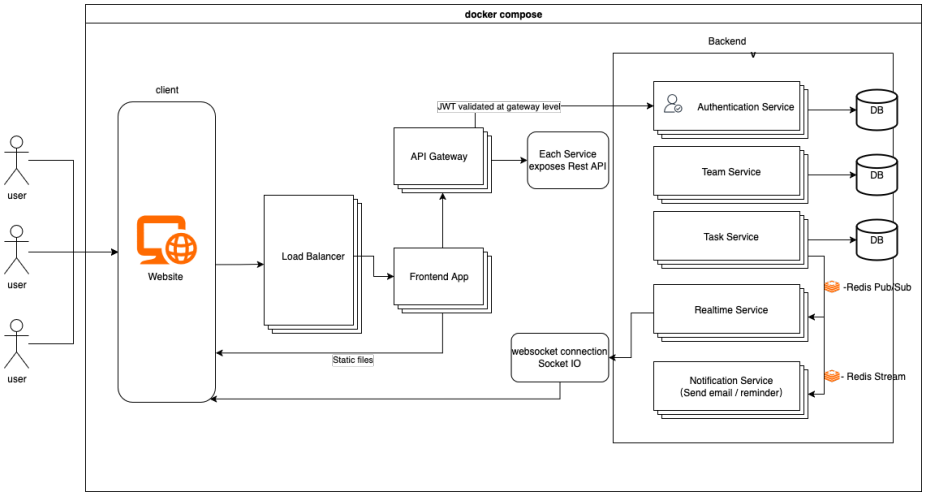

Figure 2: Original Architecture

Original Architecture The original architecture was already microservices-based but used Redis for caching and direct service communication.

1.3 System Components

- Frontend: React 18 + TypeScript SPA with WebSocket client
- API Gateway: Nginx reverse proxy with load balancing
- Services:
  - Auth Service: JWT authentication, user management
  - Team Service: Team CRUD, member management
  - Task Service: Task CRUD, event publishing
  - Realtime Service: WebSocket hub, Kafka consumer
  - Notification Service: Email notifications
  - Data: MySQL databases (one per service)
  - Message Broker: Apache Kafka

2. Data Flow Architecture

Authentication Flow

```
User → Frontend → Nginx → Auth Service → Auth DB
                  ↓
              JWT Token → Frontend Storage
```

Task Management Flow

```
User → Frontend → Nginx → Task Service → Task DB
                  ↓              ↓
              JWT Validation  Team Service (permission check)
                  ↓              ↓
              Auth Service    Team DB
```

Real-time Updates Flow

```
Task Service → Kafka → Realtime Service → WebSocket → Frontend
     ↓              ↓
Notification Service → SMTP → Email
```

Event-Driven Architecture

```
Service Actions → Kafka Events → Multiple Consumers
                       ↓

                Realtime Service  → WebSocket Updates

                       ↓

               Notification Svc   → Email Notifications
```

3. Technology Stack

Backend Services

- **Language:** Go 1.21+
- **Framework:** Gin (HTTP framework)
- **Database:** MySQL 8.0
- **Message Queue:** Apache Kafka
- **Authentication:** JWT (JSON Web Tokens)
- **Migration:** DBMate

Frontend

- **Framework:** React 18 with TypeScript

- **Build Tool:** Vite
- **Routing:** React Router
- **State Management:** React Context + Local Storage
- **Real-time:** WebSocket API
- **HTTP Client:** Fetch API

Infrastructure

- **Containerization:** Docker & Docker Compose
- **Reverse Proxy:** Nginx
- **Development:** Mailpit (SMTP testing)
- **Database Admin:** phpMyAdmin (optional)

Key Implementation Details

- Backend by Ziyi Hong: backend_overview
- Frontend by Danylo Tulainov: frontend_overview

Reflection in Backend

What Worked Well

- Microservices architecture enabled independent development and scaling
- Event-driven architecture provided good decoupling and scalability
- WebSocket implementation delivered quick real-time update
- Docker containerization simplified deployment and development

Major Challenges

- WebSocket Connection Management: Had to implement robust reconnection logic and handle browser compatibility issues
- Distributed System Complexity: Required careful error handling and monitoring across services
- Data Format and Schema Inconsistencies: The most challenging issue was handling different data formats between services

Take a bug for example:
The backend sent a time.Time object for the due field in the Kafka event payload.
However, the frontend expected a string in "YYYY-MM-DD" format.
This mismatch caused WebSocket events to fail silently. The frontend would receive the event but couldn't parse the due field, leading to incomplete task updates that required manual page refresh. The issue took so much time to debug because the WebSocket connection appeared to be working, but the data parsing was failing silently.

What I'd Do Differently

- Architecture: comprehensive monitoring (Prometheus/Grafana)
- Technology:
  - Database: consider PostgreSQL for better JSON support.
  - **Redis for caching**: The current system lacks caching, leading to repeated database queries for user authentication, team membership validation, and task data. Redis would significantly improve performance by caching frequently accessed data. It was planned, but didn't have enough time to implement it. I think there is a bottleneck in ws, because of the repeated queries for user authentication and team membership validation, with cache it will be faster and more efficient.
- Process: comprehensive testing strategy, automated deployment.
- Data Handling: Implement stricter schema validation and especially need **better error reporting** for format mismatches
- Development Process: I should have worked on frontend parallel with backend development. Less backend development and do more frontend work so that we could present the project more visually. Now even though

we have a “laufbar und umfassend” backend, we don’t have the frontend to showcase it.

Key Learnings

1. Distributed System Design Principles:
   - Service Decomposition: Breaking down monolithic applications into microservices requires careful consideration of service boundaries and data ownership
   - Event-Driven Architecture: Using events for inter-service communication provides excellent decoupling but introduces complexity in event ordering and consistency
   - Data Consistency: Achieving ACID properties across services is challenging; eventual consistency with compensation patterns is often more practical
   - Fault Tolerance: Distributed systems must be designed to handle partial failures gracefully, requiring circuit breakers, retries, and fallback mechanisms
2. Communication Patterns:
   - Synchronous vs Asynchronous: REST APIs for request-response patterns, message queues for event-driven communication
   - Service Discovery: Dynamic service location is crucial for scalability and fault tolerance
   - Load Balancing: Distributing load across multiple service instances requires careful consideration of session affinity and health checks
3. Authentication and Authorization:
   - JWT Token Management: Stateless authentication using JWT tokens enables horizontal scaling but requires careful token validation and refresh mechanisms
   - Cross-Service Authentication: Services need to validate tokens and extract user context without maintaining session state
   - WebSocket Authentication: Authenticating WebSocket connections requires special handling, such as token validation during connection upgrade
   - Internal Service Communication: Using internal tokens for service-to-service communication provides security isolation
4. Real-time Communication:
   - WebSocket Management: Maintaining persistent connections in distributed systems requires connection pooling, heartbeat mechanisms, and proper cleanup
   - Event Broadcasting: Targeting specific users or groups in real-time requires efficient user-to-connection mapping and event routing
5. Monitoring and Observability:
   - Distributed Tracing: Tracking requests across multiple services is essential for debugging and performance optimization
   - Health Checks: Implementing proper health checks enables auto-

matic failure detection and recovery
    - Centralized Logging: Aggregating logs from multiple services provides visibility into system behavior
6. Data Management:
    - Database per Service: Each service owning its data provides better isolation but requires careful design of data access patterns
    - Event Sourcing: Storing events as the source of truth enables replay and audit capabilities
    - Schema Evolution: Managing data format changes across services requires versioning and backward compatibility strategies

Backend Overview

- Reflection - Project challenges, learnings, and improvements

Architecture

The backend consists of 5 microservices communicating through REST APIs and Kafka events:

- **Auth Service** - User authentication and JWT management
- **Task Service** - Task CRUD operations and event publishing

- **Team Service** - Team management and member permissions
- **Realtime Service** - WebSocket connections and event broadcasting
- **Notification Service** - Email notifications via Kafka events

Infrastructure Components

- **Kafka** - Message broker and event streaming platform
- **Nginx Gateway** - API gateway, load balancer, and WebSocket proxy
- **MySQL Databases** - One database per service for data isolation

Communication Patterns

- **Synchronous**: REST APIs for request-response operations
- **Asynchronous**: Kafka events for real-time updates and notifications
- **Real-time**: WebSocket connections for live collaboration

Key Technologies

- **Language**: Go 1.21+ with Gin framework
- **Database**: MySQL (one per service)
- **Message Broker**: Apache Kafka
- **Authentication**: JWT tokens
- **Containerization**: Docker

Documentation

- API Endpoints - Complete API reference
- Auth Service - Authentication and user management
- Task Service - Task operations and events
- Team Service - Team management and permissions
- Realtime Service - WebSocket and real-time updates
- Notification Service - Email notifications
- Kafka - Event streaming and partitioning strategy
- Nginx Gateway - API gateway and load balancing

Key Features

- **Event-Driven Architecture**: Services communicate via Kafka events
- **Real-time Collaboration**: WebSocket-based live updates
- **Team-based Partitioning**: Kafka events partitioned by team ID for ordering
- **Horizontal Scaling**: Services designed for multiple instances
- **JWT Authentication**: Stateless authentication across services

Auth Service Documentation

Overview

The **Auth Service** handles user authentication, JWT token management, and user profile operations.
It provides JWT validation for other services and manages user data with secure password hashing.

- Auth service API doc: Auth API

Architecture

- **Port**: `8084`
- **Database**: MySQL (`3309`)
- **Dependencies**: None (standalone service)
- **Event Publishing**: Kafka for user events

Key Features

- User registration and authentication
- JWT token generation and validation
- User profile management
- Password management with **SHA-256 hashing**
- Role-based access control (user/admin)
- Internal API for token validation

Authentication

- Uses **JWT tokens** for authentication.
- Provides `/validate` endpoint for other services to verify tokens.

Event Publishing

The service publishes events to **Kafka** for real-time updates:
- `user.created` — when a user is created

Key Files

- **Main Entry**: `auth/cmd/auth-service/main.go`
- **Handlers**: `auth/internal/handlers/http.go`
- **Models**: `auth/internal/models/models.go`
- **Repository**: `auth/internal/repository/repository.go`
- **Service**: `auth/internal/service/auth_service.go`
- **Middleware**: `auth/internal/middleware/jwt.go`
- **Events**: `auth/internal/events/producer.go`
- **Migrations**: `auth/migrations/`
- **API Spec**: `auth/api/auth.yml`

Task Service Documentation

Overview

The **Task Service** handles task management operations including CRUD operations, task assignment, and real-time event publishing.
It integrates with the **Team Service** for permission validation and publishes events to **Kafka** for real-time updates.

- Task service API doc: Task API

Architecture

- **Port**: `8081`
- **Database**: MySQL (`3306`)
- **Dependencies**: Auth Service (JWT validation), Team Service (permission checks)
- **Event Publishing**: Kafka for real-time updates

Key Features

- Task CRUD operations
- Team-scoped task management
- Task assignment and completion tracking
- Real-time event publishing
- Cross-service authentication

Authentication

All endpoints require **JWT authentication** via: Authorization: Bearer

Event Publishing

The service publishes events to **Kafka** for real-time updates:
- `task.created` — when a task is created
- `task.updated` — when a task is updated
- `task.deleted` — when a task is deleted
- `task.completed` — when task completion status changes

Key Files

- **Main Entry**: `task/cmd/task-service/main.go`
- **Handlers**: `task/internal/handlers/http.go`
- **Models**: `task/internal/models/models.go`
- **Repository**: `task/internal/repository/repository.go`
- **Events**: `task/internal/events/producer.go`
- **Migrations**: `task/migrations/`
- **API Spec**: `task/api/task.yml`

Team Service Documentation

Overview

The **Team Service** handles team management operations including team CRUD, member management, and permission control.
It provides internal APIs for other services to validate team access and publishes team-related events to **Kafka**.

- Team service API doc: Team API

Architecture

- **Port**: `8083`
- **Database**: MySQL (`3307`)
- **Dependencies**: Auth Service (JWT validation)
- **Event Publishing**: Kafka for real-time updates

Key Features

- Team CRUD operations
- Team member management
- Permission control (owner, admin, member roles)
- Internal API for team access validation
- Real-time event publishing

Authentication

- Most endpoints require **JWT authentication**.
- Internal endpoints use **internal token authentication**.

Event Publishing

The service publishes events to **Kafka** for real-time updates:
- `team.created` — when a team is created
- `team.updated` — when a team is updated
- `team.deleted` — when a team is deleted
- `team.member_added` — when a member is added

- `team.member_removed` — when a member is removed
- `team.member_role_updated` — when a member's role changes

Key Files

- **Main Entry**: `team/cmd/team-service/main.go`
- **Handlers**: `team/internal/handlers/handlers.go`
- **Models**: `team/internal/models/models.go`
- **Repository**: `team/internal/repository/repository.go`
- **Events**: `team/internal/events/events.go`
- **Migrations**: `team/migrations/`
- **API Spec**: `team/api/team.yml`

Realtime Service Documentation

Overview

The **Realtime Service** manages WebSocket connections and provides real-time updates to the frontend.
It consumes events from **Kafka** and broadcasts them to connected users based on team membership and user relationships.

Architecture

- **Port**: `8086`
- **Database**: None (stateless)
- **Dependencies**: Auth Service (JWT validation), Team Service (member resolution), Kafka
- **Event Processing**: Kafka consumer for real-time updates

Key Features

- WebSocket connection management
- User-specific connection mapping
- Kafka event consumption and processing
- Real-time event broadcasting
- Team member resolution for targeted updates
- Connection health monitoring

Authentication

WebSocket connections are authenticated via **JWT tokens** passed in:
- The connection URL, or
- `Sec-WebSocket-Protocol` header

Event Processing

Consumes events from **Kafka** topics:
- `task.*` — task-related events
- `team.*` — team-related events
- `user.*` — user-related events

Broadcasts events to specific users based on:
- Team membership
- Task assignment
- Event ownership

Key Files

- **Main Entry**: `realtime/main.go`

- **WebSocket Hub**: `realtime/websocket.go`
- **Kafka Consumer**: `realtime/kafka_consumer.go`

- **Events**: `realtime/events.go`

Notification Service Documentation

Overview

The **Notification Service** handles email notifications by consuming events from **Kafka** and sending appropriate emails to users.
It provides asynchronous notification processing for task and team-related events.

Architecture

- **Port**: `8090`
- **Database**: None (stateless)
- **Dependencies**: Kafka, SMTP server
- **Event Processing**: Kafka consumer for notification triggers

Key Features

- Email notification sending
- Kafka event consumption
- SMTP integration
- Asynchronous notification processing
- Event-based notification triggers

Authentication

Uses **internal authentication** for service-to-service communication.

Event Processing

Consumes events from **Kafka** topics and sends appropriate notifications:
- `task.*` — task-related notifications
- `team.*` — team-related notifications
- `user.*` — user-related notifications

Key Files

- **Main Entry**: `notification/main.go`
- **Kafka Consumer**: `notification/kafka_consumer.go`

- **Email Sender**: `notification/email_sender.go`
- **Auth Client**: `notification/auth_client.go`

Kafka Documentation

Overview

Apache Kafka serves as the message broker and event streaming platform for the entire application.
It enables asynchronous communication between services and provides event persistence and replay capabilities.

Architecture

- **Port**: `9092`
- **Role**: Message Broker, Event Streaming Platform
- **Dependencies**: None (standalone service)
- **Storage**: Persistent event storage

Key Features

- Event streaming and message queuing
- Event persistence and replay
- Topic-based message routing
- Partitioning for scalability
- Multiple consumer support
- Fault tolerance and durability

Event Topics

- `task.created` — task creation events
- `task.updated` — task update events
- `task.deleted` — task deletion events
- `task.completed` — task completion events
- `team.created` — team creation events
- `team.updated` — team update events

- `team.deleted` — team deletion events
- `team.member_added` — team member addition events
- `team.member_removed` — team member removal events
- `team.member_role_updated` — team member role update events
- `user.created` — user creation events

Partitioning Strategy

Key Principle

All events are partitioned by **Team ID** to ensure:
- **Event Ordering**: Events for the same team are always processed in sequence
- **Scalability**: Events from different teams can be processed in parallel
- **Consistency**: WebSocket clients receive events in the correct order
- **Load Distribution**: Events are distributed across partitions according to team activity

Partition Keys

- **Task Events**: `"task:" + taskId` → routed by `teamId`
- **Team Events**: `"team:" + teamId`
- **User Events**: `"user:" + userId`

Event Flow

Events are published by services and consumed by other services via **Kafka topics**.
Events are partitioned and persisted, allowing for **replay** and **fault-tolerant processing**.

Persistence

All events are durably stored in Kafka's internal log for replay and recovery.

Key Files

- **Docker Config**: `docker-compose.yml` (kafka service)
- **Producer Examples**:
  - `task/internal/events/producer.go`

  - `team/internal/events/events.go`
  - `auth/internal/events/producer.go`
- **Consumer Examples**:
  - `realtime/kafka_consumer.go`
  - `notification/kafka_consumer.go`

Configuration

Environment Variables:
- `KAFKA_BROKERS` — Kafka broker addresses (default: `dev_kafka:9092`)
- `KAFKA_TOPIC_PREFIX` — optional topic prefix
- `KAFKA_GROUP_ID` — consumer group ID

Event Schema

All events follow a consistent JSON structure:

```
{
  "eventType": "task.created",
  "taskId": 123,
  "teamId": 456,
  "actorId": 789,
  "timestamp": "2025-01-20T10:30:00Z",
  "payload": { ... }
}
```

Benefits

- **Event Ordering**: Guaranteed ordering of events within each team
- **Scalability**: Supports parallel processing across teams and consumers
- **Fault Tolerance**: Team-level isolation prevents cascading failures
- **Performance**: Load distribution adapts to team activity

Scalability

- **Partitioning**: Events are partitioned by `teamId` to enable parallel processing across teams
- **Consumer Groups**: Multiple consumers within a group can share the processing load

- **Horizontal Scaling**: Kafka clusters can be scaled with multiple brokers for high availability and throughput

Nginx Gateway Documentation

Overview

The **Nginx Gateway** serves as the API gateway and reverse proxy for the entire application.
It handles load balancing, WebSocket proxying, static file serving, and request routing to appropriate microservices.

Architecture

- **Port**: `80`
- **Role**: API Gateway, Load Balancer, WebSocket Proxy
- **Dependencies**: All backend services
- **Static Files**: Frontend React application

Key Features

- Request routing to microservices
- Load balancing across service instances
- WebSocket proxy and upgrade handling
- Static file serving for frontend
- CORS configuration
- Health check integration

Service Routing

- **Frontend**: Serves React SPA from `/`
- **Auth Service**: Routes `/api/auth/*` → `auth-service:8084`
- **Task Service**: Routes `/api/tasks/*` → `task-service:8081`
- **Team Service**: Routes `/api/teams/*` → `team-service:8083`
- **WebSocket**: Routes `/ws` → realtime service with load balancing

WebSocket Configuration

- Supports WebSocket upgrade and connection management
- Load balances WebSocket connections across multiple realtime service instances
- Handles `Sec-WebSocket-Protocol` headers for JWT authentication
- Maintains connection state and heartbeat

Load Balancing

- **Realtime Service**: Dynamic DNS resolution for horizontal scaling
- **Other Services**: Direct service routing
- **Health Checks**: Integrated with Docker health checks

Key Files

- **Configuration**: `nginx.conf`
- **Docker Config**: `docker-compose.yml` (nginx service)

API Endpoints Overview

This document lists all current HTTP endpoints across the Auth, Team, and Task services, with auth requirements and sample curl commands.

- Base URLs:
  - Auth: http://localhost:8084
  - Team: http://localhost:8083
  - Task: http://localhost:8081
- Unless noted, endpoints return JSON. Most endpoints require Authorization: Bearer .

Swagger

You can check our API documentation in swagger editor by importing the file in it. - Task service API doc: Task API - Team service API doc: Team API - Auth service API doc: Auth API

Auth Service (localhost:8084)

- Health
  - GET /healthz
    - curl: `curl -sS http://localhost:8084/healthz`
- Internal endpoints
  - GET /internal/users/{id}
    - Internal service endpoint (no auth required for dev)
    - curl: `curl -sS http://localhost:8084/internal/users/1`
- Authentication
  - POST /auth/register
    - Body: { username, email, password, firstName?, lastName? }
    - curl:

```
curl -sS -X POST http://localhost:8084/auth/register \
  -H 'Content-Type: application/json' \
  -d '{"username":"newuser","email":"new@example.com","password":"password"}'
```

  - POST /auth/login
    - Body: { username, password }
    - curl:

```
curl -sS -X POST http://localhost:8084/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"admin","password":"password"}'
```

  - POST /auth/refresh
    - Body: { refreshToken }
    - curl:

```
curl -sS -X POST http://localhost:8084/auth/refresh \
  -H 'Content-Type: application/json' \
  -d '{"refreshToken":"$REFRESH"}'
```

  - POST /auth/logout

        * Auth required
        * curl: `curl -sS -X POST http://localhost:8084/auth/logout -H "Authorization: Bearer $ACCESS"`
- Token validation (internal utility)
  - POST /validate
    * Auth required; returns `{ valid: true, user: { id, username, role } }` if token is valid
    * curl: `curl -sS -X POST http://localhost:8084/validate -H "Authorization: Bearer $ACCESS"`
- Users
  - GET /users
    * List users (typically admin). Query: q, limit, offset
    * curl: `curl -sS http://localhost:8084/users -H "Authorization: Bearer $ACCESS"`
  - POST /users
    * Create user (admin). Body: { username, email, password, firstName?, lastName?, role }
    * curl:
      ```
      curl -sS -X POST http://localhost:8084/users \
        -H "Authorization: Bearer $ACCESS" -H 'Content-Type: application/json' \
        -d '{"username":"alice","email":"alice@example.com","password":"password","ro
      ```
  - GET /users/{id}
    * Get by ID
    * curl: `curl -sS http://localhost:8084/users/1 -H "Authorization: Bearer $ACCESS"`
  - PUT /users/{id}
    * Update fields. Body: any subset of { username, email, firstName, lastName, role, isActive }
    * curl:
      ```
      curl -sS -X PUT http://localhost:8084/users/1 \
        -H "Authorization: Bearer $ACCESS" -H 'Content-Type: application/json' \
        -d '{"firstName":"Admin","lastName":"User"}'
      ```
  - DELETE /users/{id}
    * Delete user (admin)
    * curl: `curl -sS -X DELETE http://localhost:8084/users/5 -H "Authorization: Bearer $ACCESS"`
  - GET /users/profile
    * Current user profile
    * curl: `curl -sS http://localhost:8084/users/profile -H "Authorization: Bearer $ACCESS"`
  - PUT /users/profile
    * Update current user profile. Body: { firstName?, lastName?, email? }
    * curl:
      ```
      curl -sS -X PUT http://localhost:8084/users/profile \
        -H "Authorization: Bearer $ACCESS" -H 'Content-Type: application/json' \
      ```

```
  -d '{"firstName":"Admin","lastName":"User"}'
```

- POST /users/change-password
  - Body: { currentPassword, newPassword }
  - curl:

```
curl -sS -X POST http://localhost:8084/users/change-password \
  -H "Authorization: Bearer $ACCESS" -H 'Content-Type: application/json' \
  -d '{"currentPassword":"password","newPassword":"password123"}'
```

Team Service (localhost:8083)

- Health
  - GET /healthz
    - curl: `curl -sS http://localhost:8083/healthz`
- Internal endpoints
  - GET /internal/teams/{id}/members
    - Internal service endpoint (requires internal token)
    - curl: `curl -sS -H "X-Internal-Token: $INTERNAL_TOKEN" http://localhost:8083/internal/teams/1/members`
- Teams
  - GET /teams
    - Query: q, limit, offset
    - curl: `curl -sS http://localhost:8083/teams`
  - POST /teams
    - Body: { name, description? }
    - curl:

```
curl -sS -X POST http://localhost:8083/teams \
  -H 'Content-Type: application/json' \
  -d '{"name":"Dev Team","description":"Main team"}'
```

  - GET /teams/{id}
    - Get team by ID
    - curl: `curl -sS http://localhost:8083/teams/1`
  - PUT /teams/{id}
    - Requires membership/owner (per middleware). Body: { name?, description? }
    - curl:

```
curl -sS -X PUT http://localhost:8083/teams/1 \
  -H 'Content-Type: application/json' \
  -d '{"name":"New Name"}'
```

  - DELETE /teams/{id}
    - Requires owner
    - curl: `curl -sS -X DELETE http://localhost:8083/teams/1`
- Team Members
  - GET /teams/{id}/members
    - List members
    - curl: `curl -sS http://localhost:8083/teams/1/members`
  - POST /teams/{id}/members

        * Add member. Body: { userId, role }
        * curl:
          ```
          curl -sS -X POST http://localhost:8083/teams/1/members \
            -H 'Content-Type: application/json' \
            -d '{"userId":5,"role":"member"}'
          ```
    - DELETE /teams/{id}/members/{userId}
        * Remove member
        * curl: `curl -sS -X DELETE http://localhost:8083/teams/1/members/5`
- Users → Teams
    - GET /users/{userId}/teams
        * List teams the user belongs to
        * curl: `curl -sS http://localhost:8083/users/1/teams`

Task Service (localhost:8081)

- Health
    - GET /healthz
        * curl: `curl -sS http://localhost:8081/healthz`
- Team-scoped tasks
    - GET /teams/{teamId}/tasks
        * Query: completed?, priority?, assigneeId?, q?, limit?, offset?
        * Auth required
        * curl: `curl -sS -H "Authorization: Bearer $ACCESS" "http://localhost:8081/teams/1/tasks?priority=high&limit=10"`
    - POST /teams/{teamId}/tasks
        * Body: { title, priority, due, description?, assigneeId? }
        * Auth required
        * curl:
          ```
          curl -sS -X POST http://localhost:8081/teams/1/tasks \
            -H "Authorization: Bearer $ACCESS" -H 'Content-Type: application/json' \
            -d '{"title":"Write docs","priority":"medium","due":"2025-08-30"}'
          ```
- Cross-team listing
    - GET /tasks
        * Query: teamId?, completed?, priority?, assigneeId?, q?, limit?, offset?
        * Auth required
        * curl: `curl -sS -H "Authorization: Bearer $ACCESS" http://localhost:8081/tasks`
- Single task
    - GET /tasks/{id}
        * Auth required
        * curl: `curl -sS -H "Authorization: Bearer $ACCESS" http://localhost:8081/tasks/1`
    - PUT /tasks/{id}
        * Body: any subset of { title, description, completed, priority, due, assigneeId }

        * Auth required
        * curl:
          ```
          curl -sS -X PUT http://localhost:8081/tasks/1 \
            -H "Authorization: Bearer $ACCESS" -H 'Content-Type: application/json' \
            -d '{"completed":true,"priority":"high"}'
          ```
    - DELETE /tasks/{id}
        * Auth required
        * curl: `curl -sS -X DELETE -H "Authorization: Bearer $ACCESS" http://localhost:8081/tasks/1`
- Sub-resources
    - PUT /tasks/{id}/assignee
        * Body: { assigneeId: number|null }
        * Auth required
        * curl:
          ```
          curl -sS -X PUT http://localhost:8081/tasks/1/assignee \
            -H "Authorization: Bearer $ACCESS" -H 'Content-Type: application/json' \
            -d '{"assigneeId":5}'
          ```
    - POST /tasks/{id}/complete
        * Body: { completed: boolean }
        * Auth required
        * curl:
          ```
          curl -sS -X POST http://localhost:8081/tasks/1/complete \
            -H "Authorization: Bearer $ACCESS" -H 'Content-Type: application/json' \
            -d '{"completed":true}'
          ```

Notes - Replace `$ACCESS`/`$REFRESH` with real tokens from the Auth login/refresh responses. - Error responses follow `{ code, message }` shape across services.

Frontend Documentation

Overview

The frontend of the project is a **React + TypeScript single-page application (SPA)** that provides the user interface for managing tasks, teams, and user accounts. It communicates exclusively with the backend microservices through an **Nginx reverse proxy** that routes `/api/*` requests to the appropriate services.

The main goals of the frontend are:

- Provide a clear **user-friendly interface** for login, registration, and navigation.
- Allow users to **create, update, and manage tasks** inside teams.
- Enable **team management**: creating teams, editing details, and inviting/removing members.

- Support **real-time updates** via WebSockets, so users see changes instantly.
- Respect **roles and permissions** (regular users vs admins).

Technology Stack

- **React (v18)** – component-based UI library.
- **TypeScript** – type safety and maintainability.
- **Vite** – fast development server and bundler.
- **React Router v6** – client-side routing.
- **Axios** – HTTP client with token injection and error handling.
- **WebSocket client** – for real-time task updates.
- **CSS (inline + utility styles)** – simple styling, no external CSS frameworks.

Project Structure

```
frontend/
  src/
      api/                # HTTP clients for backend services
          auth.ts
          team.ts
          task.ts
          http.ts
      layouts/            # Shared layout wrappers
          AppLayout.tsx
          AuthLayout.tsx
      pages/              # Page-level components
          LoginPage.tsx
          RegisterPage.tsx
          WelcomePage.tsx
          tasks/
              TasksPage.tsx
              DetailedTaskPage.tsx
              CompletedPage.tsx
          teams/
              TeamsPage.tsx
              CreateTeamPage.tsx
              TeamTasksPage.tsx
              TeamsDetailedPage.tsx
          admin/
              UsersAdminPage.tsx
              TeamsAdminPage.tsx
```

```
            TeamAdminDetailPage.tsx
      realtime/           # WebSocket logic
         ws.ts
         useRealtime.ts
         RealtimeRoot.tsx
      routes/             # Auth guards
         RequireAuth.tsx
         RequireAdmin.tsx
      App.tsx             # Router and app root
      main.tsx            # Entry point
   package.json
   tsconfig.json
   vite.config.ts
```

API Layer

http.ts

A thin wrapper around Axios.

- Automatically adds **Authorization** header if access token exists.
- Handles JSON encoding/decoding.
- Throws meaningful errors if the backend returns non-2xx responses.

```ts
import axios from "axios";

export const http = axios.create({
  baseURL: "/api",
  headers: { "Content-Type": "application/json" },
});
```

---

auth.ts

Handles authentication calls to `/api/auth/*`.

- `login(credentials)` – POST /auth/login.
- `register(credentials)` – POST /auth/register.
- Stores access token in `localStorage`.

---

team.ts

Handles team management.

- `createTeam(input)` – create a new team.

- `getTeamById(id)` – fetch team details.
- `listUserTeams(userId)` – get all teams for a given user.
- `listTeamMembers(teamId)` – get members of a team.
- `updateTeam(id, patch)` – change team name/description.
- `deleteTeam(id)` – delete a team.
- `adminAddMember(teamId, userId, role)` – add user by ID.
- `adminRemoveMember(teamId, userId)` – remove member.

`task.ts`

Handles task CRUD.

- `listMyTasks()` – list all tasks assigned to current user.
- `listTasksForTeam(teamId)` – list all tasks in a team.
- `createTaskInTeam(teamId, input)` – create new task.
- `getTask(id)` – fetch single task.
- `updateTask(id, patch)` – update fields of a task.
- `deleteTask(id)` – delete a task.
- `setAssignee(id, userId)` – assign task to member.
- `setCompleted(id, completed)` – mark task done/undone.

Pages

1. Authentication

- **LoginPage** – login form, saves token, redirects to `/welcome`.
- **RegisterPage** – create new account.
- **WelcomePage** – greeting page after login.

2. Teams

- **TeamsPage** – lists all teams the user is part of.
- **CreateTeamPage** – create a new team.
- **TeamTasksPage** – main view inside a team: task list, create tasks.
- **TeamsDetailedPage** – management view: rename team, delete, invite/remove members.

3. Tasks

- **TasksPage** – personal task list ("My Tasks").
- **DetailedTaskPage** – detailed editing of one task.
- **CompletedPage** – view completed tasks.

4. **Admin**

- **UsersAdminPage** – list all users.
- **TeamsAdminPage** – list all teams.
- **TeamAdminDetailPage** – inspect a team as admin.

Routing

The router is defined in `App.tsx` with React Router v6.

- `/` → Login page.
- `/register` → Register page.
- `/welcome` → Welcome dashboard.
- `/teams` → List of teams.
- `/teams/new` → Create new team.
- `/teams/:id` → Team tasks.
- `/teams/:id/manage` → Manage team (members, settings).
- `/tasks` → My tasks.
- `/tasks/:id` → Detailed task view.
- `/completed` → Completed tasks.
- `/admin/*` → Admin-only pages.

Auth guards:

- `RequireAuth` ensures token is set.
- `RequireAdmin` ensures user has admin role.

Real-Time Updates

Implemented via WebSockets:

- Client connects to `/ws` after login.
- Subscribes to events such as:
  - `task.created`
  - `task.updated`
  - `task.deleted`
  - `task.completed`
- The UI updates automatically without page reload.

UI/UX Principles

- **Minimalistic design**: clean forms, simple buttons, focus on functionality.

- **Error handling**: API errors are shown in red text.
- **Optimistic updates**: UI updates immediately, then confirmed by WebSocket.
- **Accessibility**: semantic HTML with labels.

Common Issues Encountered

- **API mismatches**: frontend expected username invites, backend required numeric IDs.
- **Nginx rewrites**: misrouted `/api/tasks/*` caused 400/405 errors.
- **Strict backend validation**: required exact JSON shape.
- **CI/CD build failures**: Docker context paths and TypeScript compile errors.
- **Time pressure**: integration of multiple microservices was complex.

Lessons Learned

- Always document backend API contracts clearly.
- Keep Nginx configuration minimal to avoid rewrite errors.
- TypeScript is powerful, but requires strict consistency in imports.
- Debugging distributed systems under time pressure is difficult — logging and monitoring are crucial.
- Start integration testing earlier to catch mismatches sooner.

Future Improvements

1. **Better Error Messages** – expose validation errors from backend to help debugging.
2. **Consistent API** – unify invite endpoints (username + ID).
3. **CI/CD Stability** – pre-build TypeScript before Docker build to avoid surprises.
4. **Improved UI** – add search, sorting, and filtering for tasks.
5. **Testing** – unit tests for components, integration tests for API layer.

Conclusion

The frontend demonstrates a **working multi-service React application** that connects to several backend services via Nginx. Despite unresolved issues, the architecture is extensible and provides a clear separation of concerns.

The project highlights both the **strengths of modern SPA development** and the **challenges of microservice integration under real-world constraints**.

Git Branching Strategy

This document proposes a Git branching strategy for structured collaboration within the project.

---

1. Motivation

In multi-module projects with separate responsibilities (e.g. Arduino, Backend, Frontend, Database), a clearly defined branching strategy helps to:

- avoid merge conflicts,
- maintain a clean and stable `main` branch,
- improve collaboration via clear workflows.

---

2. Strategy selection

The following models were evaluated:

- **Git Flow**: full-featured but complex; more suitable for long release cycles. https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow
- **GitHub Flow**: lightweight and CI/CD-friendly. https://docs.github.com/en/get-started/using-github/github-flow
- **Trunk-Based Development**: promotes short-lived branches and continuous integration. https://trunkbaseddevelopment.com/

Among these, **a combination of GitHub Flow and Trunk-Based Development** is proposed, as it allows: - short feature branches for parallel development, - the ability of integrating with CI tools - clear merge and review processes.

Rules: - All development is done on short-lived branches derived from `main`. - Each change is reviewed and merged via Pull Request. - CI checks (tests, linters) must pass before merging.

---

3. Branch naming

Type/action-module

Type	Example	Use case
`feature`	`feature/add-backend-api`	New functionality
`fix`	`fix/mqtt-reconnect`	Bug fix
`docs`	`docs/update-readme`	Documentation
`test`	`test/add-backend-tests`	Testing-related change

Type	Example	Use case
`chore`	`chore/update-gitignore`	Setup or config change

Use lowercase and dashes.

---

4. Developer workflow

1. Create a feature branch from `main`:

```
git checkout main
git pull origin main
git checkout -b feature/<task-name>
```

2. Git commit and push the branch to GitHub:

```
git commit
git push origin feature/<task-name>
```

3. Create a Pull Request in github
4. Code Review
5. After successful review and test, merge and delete the branch:

```
git branch -d feature/<task-name>
git push origin --delete feature/<task-name>
```

---

5. Merge and CI

- `main` should be a protected branch (no direct pushes)
- All changes must go through PRs
- Merges are only allowed after successful CI
- "Squash and merge" is recommended to keep history clean